Foundation for Virtual and Augmented Reality Systems

Prof. Samit Bhattacharya

Department of Computer Science and Engineering

Indian Institute of Technology, Guwahati

Week – 12

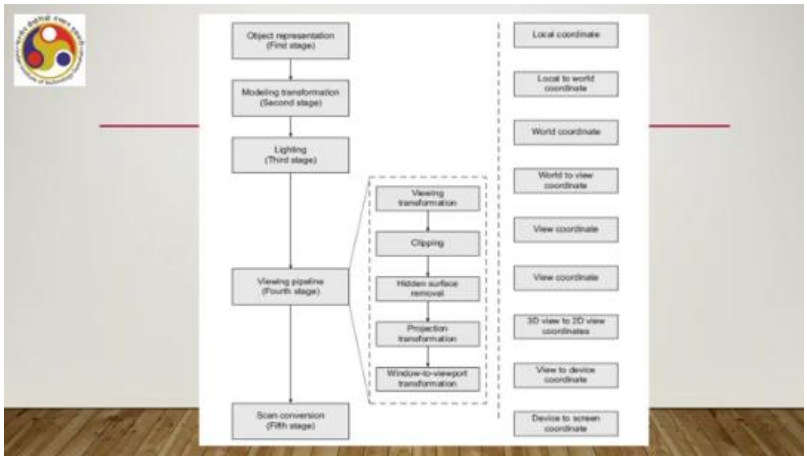
Lecture – 58

VR resource optimization

Hello and welcome to the NPTEL MOOCs course on Foundation for Virtual and Augmented Reality Systems. We will be continuing our discussion on resource optimization. In particular, we are focusing on VR resource optimization, that is, resource optimization concerns that are typically encountered when we are dealing with immersive experiences. So earlier, in this lecture, in the other parts of this lecture, we talked about the basic idea of computer graphics. We know that rendering is the most challenging part of an immersive experience generation from the point of view of the system.

So, we need to have a basic understanding of the rendering process, which is the field of computer graphics that deals with this. So, we are talking about computer graphics. Earlier we talked about the basic idea of the computer graphics field, what it is. We learned about a few terminologies, and we also introduced the 3D graphics pipeline and the stages in the pipeline. Among those stages, there are five stages as we have seen. Among those, we have covered so far four stages: the first stage, object representation; the second one, modeling transformation; the third one, lighting; and the fourth one, viewing transformation.

In this lecture, what we are going to do is basically talk about the fifth and the last stage of the graphics pipeline, and also we will be talking about how to implement the pipeline implementation aspect from both the software point of view, where implementation can be considered in the form of libraries, and from the hardware point of view, that is, how the GPUs or graphics processing units help in faster rendering. So, just to put the matter into perspective, this is the 3D graphics pipeline. This is the first stage, object representation, which takes place in the local coordinate system,



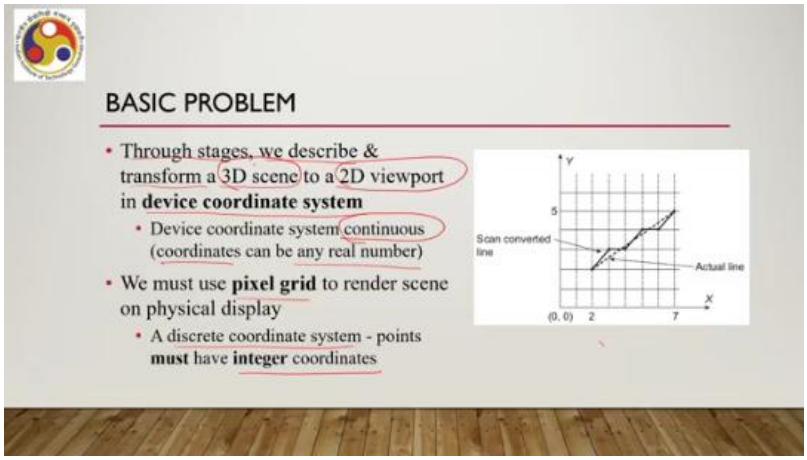
Then modeling or geometric transformation transforms from the local coordinate to the world coordinate system, then lighting, takes place in the world coordinate system. Then the view pipeline, comprising a viewing transformation that is a transformation from world to 3D view coordinate system, then a clipping operation which is a process which happens in the 3D view coordinate system. Then hidden surface removal again happens in the 3D view coordinate system.

Then projection transformation, that is transformation from 3D view coordinate to 2D view coordinate system. Then window to viewport transformation, that is transforming from 2D view coordinate system to a device coordinate system. So, today in this part of the lecture, we are going to talk about the fifth stage, that is scan conversion, where we transform the device coordinate description to a screen coordinate description that is on the pixel grid. So, this stage, stage 5, is called the rendering stage, or it may also be called the scan conversion stage.

So, all the terms are synonymous. Now, what is the basic problem that is dealt with in this stage? Now, through the four stages, what we have done so far is to describe and transform a 3D scene to a 2D viewport, which is defined on a device coordinate system. Now, when we are talking of a device coordinate system, essentially, the system is a continuous coordinate system.

That means the coordinate values can be any real number. However, when we are talking about a device or display device, we know that it has a pixel grid which is used to render the image. Now, The problem here is to map from a continuous coordinate system, that is the device coordinate system, to a discrete coordinate system where the coordinates can only be integer values. Because on a pixel grid, the points must have integer coordinates, whereas in the viewport description, points can have any real number as coordinates.

So, the objective is to transform from this continuous space to the discrete pixel grid space. The idea can be understood from this illustration here. As you can see, suppose each of these intersections represents a pixel. So, you have a line. And we want to represent it on this pixel grid.

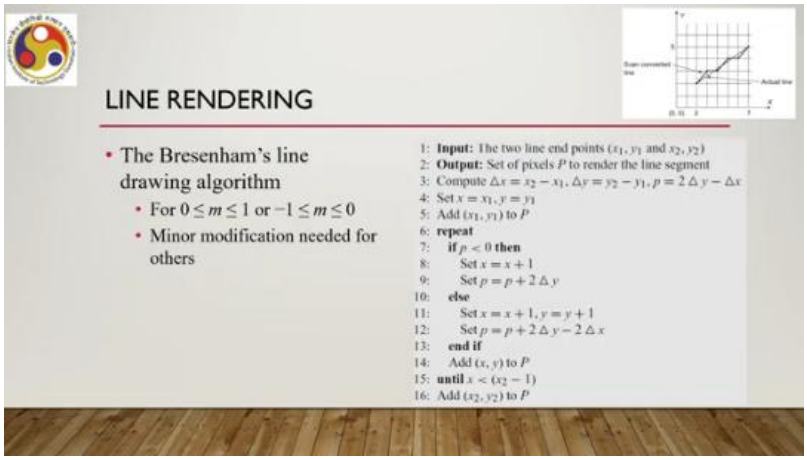


Now, as you can see, the line can pass through either a pixel or the interval between pixels. So, any real number is allowed as a coordinate. So, that is shown in the dotted line. However, when we are trying to render it on the display device, we have to choose the right pixels to color so that it represents the line.

That means we can choose, suppose, this pixel, then this one, then this one, this one, and this one instead of the points that lie in between pixels. So, essentially, we are mapping from any real value to integer values representing the nearest pixel, and this process of mapping is the fifth stage, that is rendering or scan conversion, or even it is called rasterization sometimes. So, we need to map from the viewport, that is the continuous space, to the pixel grid, that is the discrete coordinate space. And these mapping algorithms or techniques are collectively known as the rendering stage, often called the scan conversion stage or rasterization stage.

Now, how is it done? So, let us start with the simple case of point rendering. Here, it is very simple. If we have a real number as a coordinate to represent the point, then we can round off the real coordinates to the nearest integer values.

For example, 2.3 and 2.6 can be rounded up to 2 and 3, which will be the mapping from the device space to the discrete space. However, that is good for points, but not good for other shapes, such as lines, circles, and any other geometric shapes. Now, line rendering is one of the most common activities that take place in image rendering under the purview of computer graphics. So, line rendering is the basis of any image rendering.



Now if we want to render a line using a very naive intuitive method, then what happens is that we have to go for lots of calculations which are floating point operations. Now as we all know, floating point operations slow down a computer. So, the objective is to have efficient algorithms that can render a line by avoiding floating point operations. One such algorithm is Bresenham's line drawing algorithm, a quite popular and well-known algorithm. So, using this algorithm, what we want to do is find out the pixels that approximately represent the line as shown in this figure.

So, we have the dotted line as the actual line. We need to identify pixels that will represent the line. And how do we find that without taking recourse to floating point operations? So, one such popular algorithm is Bresenham's line drawing algorithm. I am just referring to this algorithm to give you some idea of what kind of computations take place in this fifth stage.

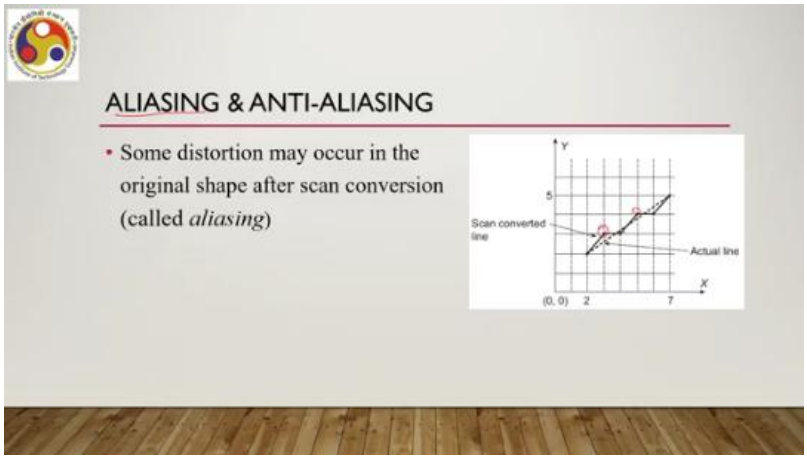
So, let us try to go through the algorithm for slopes which lie between 0 and 1. or between -1 and 0. So, effectively between -1 and +1. So, minor modification may be needed for any other slope. Now, here the input is the two line endpoints, this one and this one, and the output is the set of pixels p to render the line segment.

That is the set of pixels that approximate the line. Now, at the beginning of the algorithm, we need to compute a few parameters, namely Δx, that is the difference between the two end point x coordinates, Δy, the difference between the two end point y coordinates, and p, a decision parameter which is twice Δy - Δx. Then we start by initializing something, that is, x equal to the first endpoint, y equal to the first endpoint, and add this to the set of pixels p. Then, if the decision parameter value p is less than 0, that is this value that we have seen, then we increment x by 1 and reset the decision parameter as the older value plus twice Δy. Otherwise, we increment x by 1 and y by 1 and reset the value as p plus twice Δy minus twice delta x. And this goes on in a loop until we reach the other endpoint. So, at every loop instance, we add the computed coordinate value to the set of pixels. Now, here you can see that the entire operation is done based on integer arithmetic, addition, multiplication, and so on. So, this actually speeds up the rendering of lines, and this is one instance where we can have an efficient algorithm for faster line rendering. There are similar algorithms possible for other shapes as well.

So, this algorithm is efficient because it avoids floating point operations such as rounding off and division, and also sometimes floating point multiplication. And avoiding these operations is very important to speed up the rendering because, at any given instant, we need to render a large number of lines or line segments simultaneously. So, speeding up certainly makes the system more efficient. Now, this algorithm we need to invoke for every line segment, and there may be thousands at any instant. For interactive graphics, where we can change the display based on user input such as in virtual reality, the computational need increases manifold because with every input, we need to change the scene, which means again redrawing hundreds of thousands of lines. So, such algorithms exist for other geometric shapes. Of course, those are a little bit more complex as compared to the line rendering shapes, such as circle, ellipse, fill area, or other polygons. We will skip those. We will not go into the details of those algorithms, and you may refer to the reference mentioned at the end.

For more details, the basic idea is that we need some algorithm to efficiently render it on the screen. That means efficiently mapping the original shape to the set of pixels that will represent it on the screen. And our objective is to do it as fast as possible. That means reducing the floating point operations as much as possible. So for a line, we can completely eliminate them.

But for other shapes, we may not be able to completely eliminate them, but we can minimize the number of operations. Now, there is one more concept we need to keep in mind, which is the aliasing effect. Remember that we are trying to get a set of pixels, such as in this line, which approximate the original line shown here in this figure. The original line is shown as a dotted line. Whereas the approximated line is shown as the thick continuous line.



Now, as you can see, the approximate line does not exactly look like the original line because we are not exactly rendering the points on the line, but rendering pixels that are closer to those points on the line or closest to those points on the line. So, the shape may get distorted. So, some distortion may occur in the original shape after scan conversion; this is called aliasing. Now, what we need are additional operations to remove or reduce such distortion so that the end effect is better.

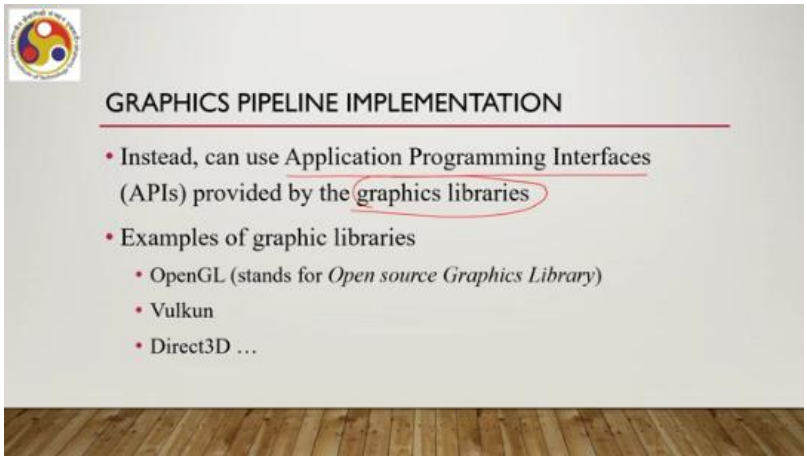
Now, these operations collectively are known as anti-aliasing operations. So, there again, some sort of complex algorithms are there to take care of, or rather minimize, the effect of aliasing. If we do not do anything, then this aliasing may lead to distortion which will be visible with change in resolution, and that will certainly reduce the quality of experience. So, anti-aliasing is essential, and it introduces even more computations to the overall computation of pixel intensities along with the other stages.

So, the previous discussion was only meant to give some idea of the amount of computations. Of course, here we skipped actual things at many places to keep things simple. We ignored many things particularly relevant for VR because, in VR, we are interested in a realistic experience where the complexity of the rendering is much more as compared to what we have seen in our discussion. Also, the scene is dynamic; it is changing based on user input, and those things we have not taken care of or discussed in the preceding discussion on the 3D graphics pipeline.

So together all of these increase the computational complexity even more. So, in VR we require rich realistic content, and a simple lighting model may not be a good option. A high degree of interaction is present in VR, which we have not taken into account in our 3D graphics pipeline discussion, and there will be motion which is achieved through animation or video. So, all these together increase the computation for rendering in a VR environment manifold, and clearly that is the challenge: how to take care of such a computationally complex rendering process without letting the user perceive any lag or distortion. That is what we are interested in doing. So, from the preceding discussion, we got some idea of the computational complexity involved. Now, let us shift our focus to the implementation of such graphics pipelines for the benefit of the developer. Earlier, we outlined the theoretical background of computer graphics.

Now, a programmer, a CG programmer, need not always implement the stages of the pipeline; that would be too much effort. Instead, the programmer can use APIs, application programming interfaces, provided by graphics libraries. So, there are already some implementations in the form of libraries.

Programmers can import those libraries into the programming environment and use the APIs provided in those libraries to take care of the graphics-related concerns. Examples of such graphics libraries are OpenGL, which stands for Open Graphics Library, Vulkan, Direct3D, and so on. Now, these APIs are predefined sets of functions which, when invoked with appropriate arguments or parameters, perform specific tasks. So, these APIs eliminate the need to know every detail of the underlying system or platform, such as the processor details, memory details, OS details, and operating system details, and so on, to build a graphics application.



If we are trying to build everything from scratch, we must have knowledge of these details, the hardware details on which our graphic system will work. However, that would be too much work every time somebody wants to develop a graphic application. Instead, they can take recourse to libraries and use the APIs provided in the libraries to deal with the lower-level tasks involved in computer graphics. For example, let us consider this OpenGL function glColor3frgb. This function is used to assign colors to a 3D point.

Here, the programmer does not require to know the details such as how color is defined in the system, how many bits are there, what are the minimum values, maximum values, and all those things. how such information is stored and accessed, what kind of hardware support is provided to store it in the form of memory organization, and how to access that, what kind of instructions are provided to access that content of the memory, and how the OS manages the call to that memory. All these details are not needed to be known by the programmer to assign color to a point if the programmer chooses to use a library.

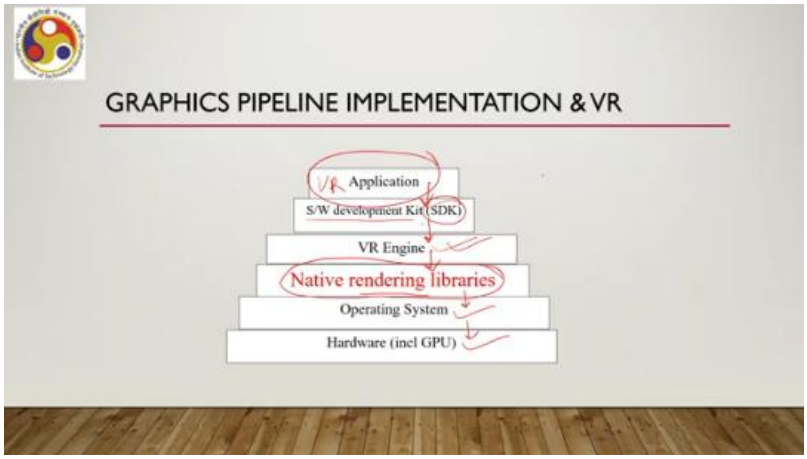
Another piece of information required is which processor handles the task, whether CPU or GPU, and accordingly, instructions need to be executed. But then, the programmer does not need to know all these details. He can simply use this function, and the function takes care of all these details and returns the color to the programmer, the desired color to the programmer. That means to the application the programmer is using to call the function.

So, that is how a programmer can think of implementing the pipeline, not by starting from scratch, but by knowing about different graphics libraries and knowing about the API supported by those libraries, and then choosing the one that suits the requirements and then using the APIs provided in that library. Now, let us try to put it in the perspective of the overall virtual reality system. Now, if we try to understand the system in a layered architecture form, then we have, at the top, the VR application layer. Below that is the software development kit or SDK layer.

Below it will be the VR engine or the game engines. So, when I say this layer, that means the application layer calls the SDK layer, and the SDK layer calls the VR engine layer. Now, below the VR engine layer lie the native rendering libraries, such as OpenGL or Vulkan or Direct3D, such libraries. The VR engine, in turn, calls the library APIs to perform the rendering activities, and below it is the operating system layer and finally the hardware layer, including the GPU. So, essentially, if a VR system developer wants to

Do something with the graphic system, then the developer does not need to bother much if he has access to such a layered system where he can take recourse to software development kits which access the VR engine. So the developer has to install the engine and import the development kit within the engine. Then whatever graphical operation is done, those are done through the APIs provided in the native rendering library. The job of the VR engine is to let the developer deal with graphics through those APIs without letting the developer directly access the native rendering libraries.

And, of course, the native libraries sit on top of the operating system, which sits on top of the hardware layer. So, this is how we can visualize the implementation and use of graphics libraries in the context of VR system development, VR system deployment, or, in other words, immersive experience. This is the software side; that is how we can implement the graphic functions or graphic operations. On the hardware side, let us try to understand in a little more detail how GPUs are useful

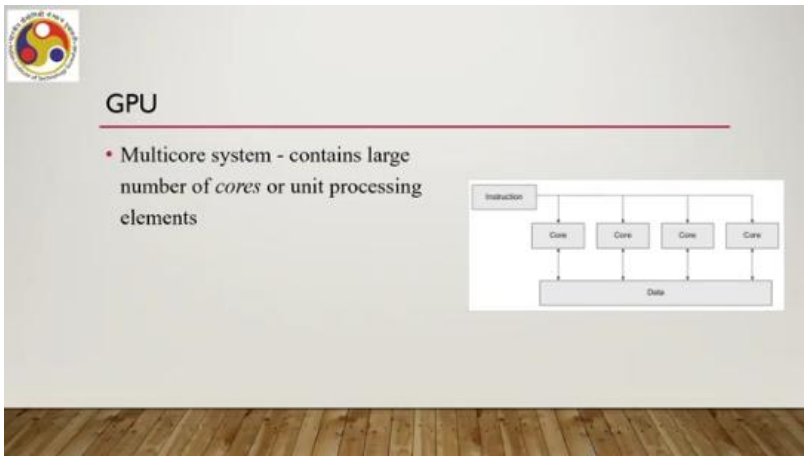


on the hardware side to make this process more efficient, to make this process faster. So, let us have a look at the role of the GPU in computer graphics in general and how that helps in virtual reality immersive experience in particular. So, the first thing that we should notice is the need for parallelism. Now, graphics operations are highly parallel. For example, modeling transformation: here what happens is that we apply transformations, such as rotation, to vertices; transformations in the form of matrices to the vertices.

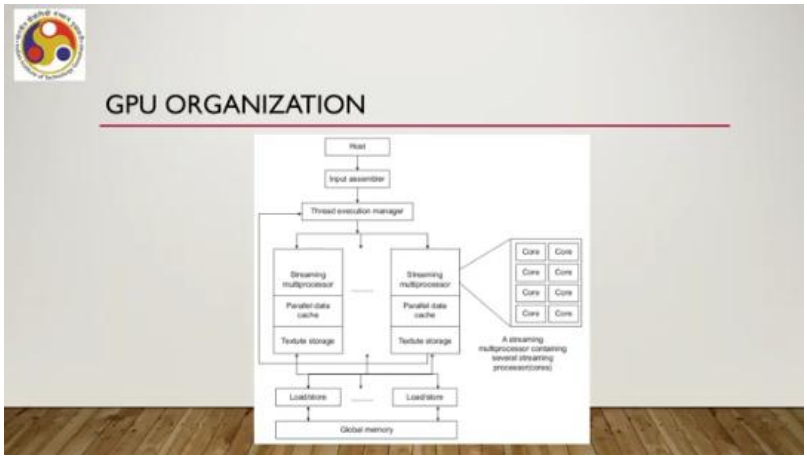
Now, a transformation is essentially a multiplication of a transformation matrix with the vertex vector. And this same vector-matrix multiplication is done for all vertices together. And ideally, we want to do it simultaneously. So, there is a parallelism involved here. So, we can have a significant gain in performance if we perform such multiplications on all vectors at the same time.

If it is not done at the same time, then it will be much, much slower, and that significant gain is important in real-time rendering because millions of vertices need to be processed per second. The number may be even more when we are talking of a realistic immersive experience. Now, CPUs generally cannot take advantage of this inherent parallelism. Traditionally, the way CPUs used to be made, because they are not designed to do that.

Of course, modern CPUs come with some such features, but they are traditionally not very good at taking care of large-scale parallel activities. So, modern graphics systems come with a separate graphics card, as we have already discussed earlier, containing their own processing unit and memory elements. And this processing unit is called a graphics processing unit or GPU, which is designed to deal with the inherent parallelism in graphics operations. Now, GPUs are typically multicore systems; that means they contain a large number of cores or unit processing elements, as shown here. You can see multiple cores are there in a GPU; they work on



Single instructions: all the cores work simultaneously on an instruction and produce data simultaneously. So, each core is capable of performing simple integer and floating-point arithmetic operations only, and multiple cores group together to form what are called streaming multiprocessor units. So, we can have Broadly, we can view a GPU as organized in this way, as shown in this figure. We have the host, then input assemblers and a thread execution manager; then this manager allocates to different streaming multiprocessors.



There can be a large number of such multiprocessors. which has a parallel data cache where the multiprocessor can produce its data. There is an element called texture storage, which essentially stores some information about predefined colors, surface colors, or predetermined surface colors. Then, these elements are accessed through the load and store

information. The elements can access whatever data is produced; it can be sent to the global memory through the load information or store information, or

the global memory content may be sent to this or accessed by these units through the load information. And each streaming multiprocessor has multiple cores, as shown here. So, a streaming multiprocessor contains several streaming processors or cores. So, how does it work? Now, most graphic systems assume a scene is made of triangles.

The scene that we are trying to render is made of triangles. That means surfaces are converted to meshes, triangular meshes. Remember our discussion on object representation, where we said that eventually everything gets converted to meshes, irrespective of how a developer defines an object. So, the graphics processing unit assumes that the mesh is available as input.

Now, through the APIs supported in the graphics library, triangles are sent to the GPU one vertex at a time. So, graphics libraries come with APIs using which such triangles in the mesh can be sent to the GPU one vertex at a time, and the GPU assembles vertices into triangles. Then the GPU performs modeling transformations on the vertices. So, multiple vertices are simultaneously transformed by multiplying the same matrix, the same transformation matrix, to all the set of vertices. The output is a stream of triangles, which are the transformed triangles from local coordinate to world coordinate.

Next, the GPU computes the vertex color. Now, the color can be computed by vector dot products, as we have demonstrated during our discussion on the simple lighting model. So, the GPU performs such dot products for multiple vertices together simultaneously. After that, each colored vertex is projected onto the view plane; that is the projection transformation, which involves the viewing transformation and projection transformation together.

So, the output is the stream of triangles in the screen or device coordinates ready to be converted to pixels. So, this projection includes all three transformations: view transformation, projection transformation, and viewport transformation. And the composite matrix is multiplied with vertices to get this final stream of triangles in the screen or device coordinates, which is a continuous coordinate space. Now, each device space triangle overlaps some pixels on the screen. In this rasterization stage, these pixels are determined, as we have discussed in stage 5.

Now, there are many rasterization algorithms. All exploit one crucial observation: that is, each pixel can be treated independently from all other pixels. Again, that is inherent parallelism. So, this leads to handling all pixels in parallel. So, given the device space triangles, we can determine the color of all the pixels simultaneously.

Now, during the pixel processing stage, two more activities take place: surface texturing and hidden surface removal. The simplest texturing method is to texture images that are already captured and stored, and apply the texture images to the geometry that we are trying to render. That means we simply replace the computed color with the color of the texture at the corresponding texture point. So, the pixel color is replaced by the texture color; that is the simplest method.

In order to do that, GPUs store textures in high-speed memory. So, each pixel calculation must access it. Now, access is very regular; that means nearby pixels tend to access nearby texture image locations, and specialized memory caches are there to reduce the access time. Note that here we have this texture storage as part of the graphics card. Also, for hidden surface removal, GPUs implement a special algorithm called the

depth buffer algorithm, which requires another storage that is called the depth buffer. So, GPUs contain such a depth buffer as a dedicated memory, which stores the depth information for each point in a 3D scene before they are projected onto a 2D plane. Depth is with respect to the pixel. Now, this memory, the depth buffer memory, stores distances of the viewer from each pixel. So, that is how GPUs work, and as you can see, GPUs are designed to take care of inherent parallelism; all the components that are there in the GPU are designed in that way, so

that parallelly, the operations can be applied on all the vertices or a large number of vertices simultaneously to speed up things. So, in summary, what we have learned is that we have learned the overall 3D graphics pipeline, which is for static images, its stages, also learned how to implement the pipeline in software via libraries, and how GPUs can support the pipeline implementation by exploiting the inherent parallelism in the pipeline stages. In the next part of the lecture, we are going to talk about how all these ideas are put into practice, how all this knowledge helped us determine the complexity of a problem involved, and

how we can try to address the complexities by taking recourse to different technologies. So, I hope you could follow the topic. Some parts, of course, we did not discuss in detail. So, I again urge you to go to the reference material for more details for a better understanding. However, our overall purpose was to give you some idea of the kind of computations involved and how we can utilize that knowledge in building the system.

We have seen for libraries how we can place them so that the life of a developer becomes easier and the pipeline stages are implemented, without the developer needing to know every nitty-gritty of a graphics system. Also, we have briefly touched upon how GPUs help in the efficient implementation of the graphics pipeline. So, looking forward to meeting you again in the next part of the lecture. Until then, thank you, goodbye, and happy learning.